



HCMUS

Viet Nam National University
Ho Chi Minh City
University of Science

fit@hcmus

Project 2 - Hashiwokakero

CSC14003: Introduction to Artificial Intelligence

Supervisor:

GS.TS. Lê Hoài Bắc

TA. Nguyễn Thanh Tình

Group 5

Full Name	Student ID
Thái Gia Huy	23120008
Nguyễn Thanh Khôi	23120009
Phan Trọng Đài	23120026
Võ Thành Nhân	23120150



Ho Chi Minh City, Ngày 21 tháng 12 năm 2025

Mục lục

1	General Information	3
1.1	Work Assignment & Contribution	3
1.2	Project Resources	3
2	Problem Analysis and Solution Description	4
2.1	Problem Definition	4
2.2	Logical Variables	5
2.2.1	Definition	5
2.3	CNF Constraints	6
2.3.1	Variable Consistency Constraints	6
2.3.2	Crossing Constraints	6
2.3.3	Island Degree Constraints	7
2.3.4	Global Connectivity Constraints	7
2.4	Example	8
2.4.1	Input	8
2.4.2	Variables	8
2.4.3	CNF Clauses	9
2.4.4	Output	9
3	Algorithm Implementation	9
3.1	Input, Ouput and Checker	9
3.1.1	Input processing	10
3.1.2	Problem representation	10
3.1.3	Output and State management	10
3.1.4	Constraint checking	11
3.2	CNF Generation and PySAT Solver	11
3.2.1	Automated CNF Encoding	11
3.2.2	PySAT Solver & Lazy Constraints	12
3.3	A* Search Algorithm	13
3.3.1	Deficit Heuristic	14
3.3.2	Connectivity Component Heuristic	15
3.3.3	Local Bottleneck Heuristic	16
3.3.4	Composite Heuristic	18
3.3.5	Forward checking	18
3.3.6	Complexity Analysis	19
3.4	Baseline Algorithms	20
3.4.1	Brute force	20
3.4.2	Backtracking	20
4	Experiments and Analysis	22
4.1	Experimental Setup	22
4.1.1	Test case	22
4.1.2	Difficulty Configurations	23
4.2	Performance Metrics	23
4.3	Scalability Analysis	24
4.3.1	Impact of Number of Islands	24
4.3.2	Impact of Grid Size	28

4.4	Heuristic Efficiency & Overall Performance	28
4.4.1	Comparison of Heuristics	28
4.4.2	Performance Heatmap	30
5	Conclusion	31
6	Tài liệu	31

1 General Information

1.1 Work Assignment & Contribution

Chi tiết phân công nhiệm vụ và mức độ hoàn thành của từng thành viên được trình bày trong Bảng 1.1:

Bảng 1.1: Bảng phân công nhiệm vụ và mức độ hoàn thành

Mục	Nhiệm vụ	Phụ trách	Mức độ hoàn thành (%)
Logic-based Approach	Phân tích bài toán và định nghĩa biến Logic	Đài	100
	Xây dựng các ràng buộc CNF (Connectivity, Bridges, etc.)	Đài	100
	Tích hợp thư viện PySAT và giải quyết bài toán	Đài	100
Search-based Approach	Cài đặt thuật toán Brute Force và Backtracking	Nhân	100
	Cài đặt thuật toán A* Search	Khôi	100
	Nghiên cứu và cài đặt các Heuristic	Khôi, Nhân	100
	Tối ưu hóa cấu trúc dữ liệu và quản lý bộ nhớ	Huy	100
Experiments & Analysis	Xây dựng bộ sinh Test Case	Nhân	100
	Chạy thực nghiệm so sánh	Huy	100
	Phân tích kết quả, vẽ biểu đồ và so sánh Heuristic	Huy, Nhân	100
Report Writing	1. General Information	Huy	100
	2. Problem Analysis (Logic)	Đài	100
	3. Algorithm Implementation	Nhân, Khôi	100
	4. Experiments & Analysis	Huy	100
	5. Conclusion	Khôi	100
Final Deliverables	Quản lý mã nguồn trên Github, viết README	Nhân, Đài	100
	Quay video demo	Huy	100

1.2 Project Resources

Các tài nguyên chính của đồ án bao gồm:

1. **Báo cáo tổng kết:** Trình bày chi tiết về cơ sở lý thuyết, phương pháp tiếp cận, quá trình cài đặt và phân tích kết quả thực nghiệm.
2. **Mã nguồn:** Được viết bằng Python, sử dụng thư viện PySAT cho phần Logic và các thư viện chuẩn cho phần Search. Cấu trúc dự án được tổ chức theo hướng module hóa để thuận tiện cho việc mở rộng và kiểm thử.
 - Repository: <https://github.com/BFCmath/Hashiwokakero-CSC14003/>
3. **Video Demo:** Video minh họa chi tiết quy trình vận hành đồ án: giới thiệu cấu trúc mã nguồn, thực thi benchmark so sánh hiệu năng giữa các thuật toán (SAT, A*, Backtracking, Bruteforce) và trực quan hóa kết quả phân tích thông qua biểu đồ và hình ảnh bàn cờ.
 - Link: <https://youtu.be/R16t57I8pfI>

2 Problem Analysis and Solution Description

2.1 Problem Definition

Hashiwokakero (hay gọi tắt là *Hashi*) là một trò chơi giải đố logic được chơi trên một lưới hình chữ nhật kích thước $m \times n$. Mặc dù lưới thường không được vẽ ra cụ thể, các ô trên lưới đóng vai trò định vị cho các thành phần của trò chơi.

Đầu vào (Input): Một trạng thái lưới ban đầu chứa các ô trống và các ô chứa số (được gọi là các “đảo” – islands). Các số này nằm trong khoảng từ 1 đến 8, biểu thị số lượng cầu cần thiết phải kết nối vào đảo đó.

Mục tiêu (Goal): Người chơi (hoặc thuật toán) cần vẽ các đường thẳng (*bridges*) kết nối các đảo với nhau sao cho thỏa mãn toàn bộ các ràng buộc của trò chơi.

Các ràng buộc (Constraints): Dựa trên luật chơi tiêu chuẩn, một lời giải hợp lệ phải tuân thủ các quy tắc sau:

1. **Kết nối hợp lệ:** Các cây cầu phải bắt đầu và kết thúc tại các đảo khác nhau, di chuyển theo đường thẳng nằm ngang hoặc thẳng đứng.
2. **Không giao cắt:** Cầu không được đi xuyên qua các đảo khác hoặc cắt chéo các cây cầu khác.
3. **Hướng đi:** Cầu chỉ được chạy theo phương vuông góc (ngang hoặc dọc), không được đi chéo.
4. **Giới hạn số lượng cầu:** Giữa hai đảo bất kỳ được kết nối trực tiếp chỉ được phép có tối đa 2 cây cầu (cầu đơn hoặc cầu đôi).
5. **Ràng buộc bậc (Degree Constraint):** Tổng số cầu kết nối với một đảo phải bằng chính xác con số ghi trên đảo đó. Ví dụ: Đảo mang số 3 phải có đúng 3 cầu nối vào nó.
6. **Tính liên thông (Connectivity):** Tất cả các đảo phải được kết nối thành một nhóm liên thông duy nhất (single connected component), tức là từ bất kỳ đảo nào phải đi được đến tất cả các đảo khác thông qua hệ thống cầu.



Hình 2.1: Một ví dụ về câu đố Hashiwokakero (trái) và một trong hai lời giải của nó (phải).

Một ví dụ về câu đố Hashiwokakero và một trong hai lời giải của nó được minh họa trong Hình 2.1.

Trong phạm vi dự án, bài toán được mô hình hóa dưới dạng một bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem), sử dụng logic mệnh đề (CNF) để tìm kiếm lời giải khả thi.

2.2 Logical Variables

Để chuyển đổi bài toán Hashiwokakero về dạng bài toán thỏa mãn mệnh đề (SAT), bài toán được mô hình hóa dưới dạng một đồ thị vô hướng, trong đó mỗi đảo tương ứng với một đỉnh, và mỗi hành lang khả dĩ nối trực tiếp hai đảo theo phương ngang hoặc dọc tương ứng với một cạnh tiềm năng.

Gọi C là tập hợp tất cả các hành lang (corridors) hợp lệ, tức là các đoạn thẳng nối hai đảo mà giữa chúng không có đảo khác hay vật cản. Với mỗi hành lang $c \in C$, số cầu có thể đặt trên đó thuộc tập $\{0, 1, 2\}$.

2.2.1 Definition

Để biểu diễn các trạng thái trên trong logic mệnh đề, với mỗi hành lang $c \in C$, nhóm định nghĩa ba biến Boolean sau:

1. Biến cầu đơn (Single Bridge Variable) Ký hiệu: $x_{c,\text{single}}$

- $x_{c,\text{single}} = \text{True}$ khi và chỉ khi có đúng một cây cầu trên hành lang c .
- $x_{c,\text{single}} = \text{False}$ trong các trường hợp còn lại.

2. Biến cầu đôi (Double Bridge Variable) Ký hiệu: $x_{c,\text{double}}$

- $x_{c,\text{double}} = \text{True}$ khi và chỉ khi có đúng hai cây cầu trên hành lang c .
- $x_{c,\text{double}} = \text{False}$ trong các trường hợp còn lại.

3. Biến kích hoạt (Active Variable – biến phụ trợ) Ký hiệu: $x_{c,\text{active}}$

- $x_{c,\text{active}} = \text{True}$ khi hành lang c có ít nhất một cầu.
- $x_{c,\text{active}} = \text{False}$ khi không có cầu nào.

Biến $x_{c,\text{active}}$ không mang thông tin mới về số lượng cầu, nhưng đóng vai trò quan trọng trong việc mã hóa các ràng buộc hình học (đặc biệt là ràng buộc giao cắt) và ràng buộc liên thông toàn cục.

Không gian biến. Nếu đồ thị có $|C| = N$ hành lang khả dĩ, tổng số biến logic cơ bản là $3N$. Việc tách riêng trạng thái “cầu đơn” và “cầu đôi” giúp đơn giản hóa việc mã hóa các ràng buộc tổng trọng số tại đảo, tránh phải sử dụng biểu diễn nhị phân phức tạp.

Không gian tìm kiếm Với $N = |C|$ hành lang, mỗi hành lang có 3 trạng thái khả dĩ $\{0, 1, 2\}$, do đó không gian tìm kiếm thô có kích thước:

$$|\Omega| = 3^N.$$

Đây là không gian tìm kiếm mà các thuật toán vét cạn hoặc backtracking phải đối mặt trong trường hợp xấu nhất.

2.3 CNF Constraints

Dựa trên các biến logic đã định nghĩa, bài toán Hashiwokakero được chuyển đổi thành một tập hợp các mệnh đề (clauses) trong dạng chuẩn hội (Conjunctive Normal Form - CNF). Một công thức CNF Φ là hội của các mệnh đề:

$$\Phi = \bigwedge_i C_i,$$

trong đó mỗi mệnh đề C_i là tuyển của các biến hoặc phủ định của biến. Để đảm bảo tính đúng đắn của lời giải, nhóm xây dựng bốn nhóm ràng buộc chính.

2.3.1 Variable Consistency Constraints

Với mỗi hành lang $c \in C$, các biến liên quan phải thỏa mãn tính nhất quán logic.

(1) Tính loại trừ. Một hành lang không thể đồng thời có một và hai cầu:

$$\neg x_{c,\text{single}} \vee \neg x_{c,\text{double}}.$$

(2) Định nghĩa biến kích hoạt. Biến $x_{c,\text{active}}$ đúng khi và chỉ khi hành lang có cầu:

$$x_{c,\text{active}} \leftrightarrow (x_{c,\text{single}} \vee x_{c,\text{double}}),$$

được chuyển sang CNF bằng ba mệnh đề:

$$\neg x_{c,\text{single}} \vee x_{c,\text{active}}, \quad \neg x_{c,\text{double}} \vee x_{c,\text{active}}, \quad \neg x_{c,\text{active}} \vee x_{c,\text{single}} \vee x_{c,\text{double}}.$$

2.3.2 Crossing Constraints

Theo luật Hashiwokakero, hai cây cầu không được giao nhau. Với mỗi cặp hành lang c_h (ngang) và c_v (dọc) giao nhau tại cùng một ô lưới, ta áp đặt:

$$\neg x_{c_h,\text{active}} \vee \neg x_{c_v,\text{active}}.$$

Ràng buộc này đảm bảo rằng không thể đồng thời kích hoạt cả hai hành lang giao nhau.

2.3.3 Island Degree Constraints

Với mỗi đảo I , gọi k_I là số cầu yêu cầu và $N(I)$ là tập các hành lang kề với đảo đó. Tổng số cầu nối vào đảo phải bằng đúng k_I :

$$\sum_{c \in N(I)} (1 \cdot x_{c,\text{single}} + 2 \cdot x_{c,\text{double}}) = k_I.$$

Do CNF không hỗ trợ trực tiếp các ràng buộc số học, đẳng thức trên được tách thành hai ràng buộc:

$$\sum \geq k_I \quad \text{và} \quad \sum \leq k_I.$$

Các biến $x_{c,\text{double}}$ được quy đổi thành hai đơn vị trong các ràng buộc đếm (cardinality constraints), đảm bảo tổng trọng số chính xác.

2.3.4 Global Connectivity Constraints

Một thách thức lớn trong mô hình SAT cho Hashiwokakero là ràng buộc rằng tất cả đảo phải thuộc một thành phần liên thông duy nhất. Mã hóa tĩnh thường dẫn đến số lượng mệnh đề rất lớn.

Do đó, ta dùng **sinh ràng buộc lười (Lazy Constraint Generation)**: chỉ thêm ràng buộc liên thông khi phát hiện nghiệm hiện tại bị ngắt quãng.

Cơ sở toán học. Xét đồ thị $G = (V, E_{\text{potential}})$. Sau một lần SAT, ta thu được tập cạnh kích hoạt E_{active} . Nếu đồ thị:

$$G' = (V, E_{\text{active}})$$

không liên thông, nó sẽ phân thành các thành phần $C_1, C_2, \dots, C_k$ với $k > 1$.

Để liên thông, phải tồn tại cạnh nối giữa C_i và $V \setminus C_i$. Tập cạnh tiềm năng như vậy gọi là tập cắt:

$$\text{Cut}(C_i, V \setminus C_i) = \{c \in E_{\text{potential}} \mid \exists u \in C_i, v \notin C_i : c \text{ nối } (u, v)\}.$$

Mệnh đề cắt (Cut Clause). Để loại bỏ nghiệm phân tách này:

$$\bigvee_{c \in \text{Cut}(C_i, V \setminus C_i)} x_{c,\text{active}}.$$

Quy trình giải.

1. Khởi tạo: xây dựng Φ chỉ từ các ràng buộc cục bộ.
2. Giải SAT: nếu UNSAT, kết luận không có lời giải.
3. Kiểm tra liên thông.
4. Nếu chỉ có 1 thành phần: nghiệm đúng, kết thúc.
5. Nếu $k > 1$: với mỗi C_i , thêm mệnh đề cắt vào Φ .
6. Lặp lại từ bước 2.

Chứng minh hội tụ. Tính đúng đắn: Bất kỳ nghiệm nào với $k = 1$ là liên thông, và các mệnh đề cắt không loại bỏ nghiệm đúng.

Tính đầy đủ: Không gian $2^{|E_{\text{potential}}|}$ hữu hạn. Mỗi mệnh đề cắt loại bỏ ít nhất một nghiệm không liên thông. Do vậy thuật toán dừng sau hữu hạn bước: hoặc tìm nghiệm liên thông, hoặc Φ trở nên mâu thuẫn.

2.4 Example

Phần này trình bày một test case nhỏ để minh họa đầy đủ quy trình: *đầu vào* $\rightarrow$ *tạo biến* $\rightarrow$ *sinh CNF* $\rightarrow$ *giải SAT* và *xuất lời giải*.

2.4.1 Input

Xét lưới 3×3 sau (0 là ô trống, số dương là đảo và bậc yêu cầu):

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Các đảo tương ứng là:

- I_1 tại $(0, 0)$ với $k_{I_1} = 1$,
- I_2 tại $(0, 2)$ với $k_{I_2} = 2$,
- I_3 tại $(2, 2)$ với $k_{I_3} = 1$.

Từ lưới này, hệ thống tự động phát hiện 2 hành lang khả dĩ:

- c_1 : nối $I_1 \leftrightarrow I_2$ theo phương ngang (đi qua ô $(0, 1)$),
- c_2 : nối $I_2 \leftrightarrow I_3$ theo phương dọc (đi qua ô $(1, 2)$).

2.4.2 Variables

Với mỗi hành lang c , mô hình sử dụng bộ ba biến: $x_{c,\text{single}}, x_{c,\text{double}}, x_{c,\text{active}}$. Ánh xạ ID theo chuẩn DIMACS (ví dụ minh họa):

ID	Ý nghĩa biến
1	$x_{c_1,\text{single}}$
2	$x_{c_1,\text{double}}$
3	$x_{c_1,\text{active}}$
4	$x_{c_2,\text{single}}$
5	$x_{c_2,\text{double}}$
6	$x_{c_2,\text{active}}$

Bảng 2.1: Các biến ngữ nghĩa cho hai hành lang trong test case.

(Lưu ý: ngoài 6 biến trên, bộ mã hoá ràng buộc bậc đảo bằng cardinality encoding sẽ sinh thêm các biến phụ trợ.)

2.4.3 CNF Clauses

(1) Ràng buộc nhất quán biến cho c_1 .

$$(\neg x_{c_1,s} \vee \neg x_{c_1,d}) \wedge (\neg x_{c_1,s} \vee x_{c_1,a}) \wedge (\neg x_{c_1,d} \vee x_{c_1,a}) \wedge (\neg x_{c_1,a} \vee x_{c_1,s} \vee x_{c_1,d})$$

Dạng DIMACS tương ứng:

$$(-1 \vee -2), (-1 \vee 3), (-2 \vee 3), (-3 \vee 1 \vee 2).$$

(2) Ràng buộc nhất quán biến cho c_2 .

$$(-4 \vee -5), (-4 \vee 6), (-5 \vee 6), (-6 \vee 4 \vee 5).$$

(3) Ràng buộc bậc đảo (minh họa). Với đảo I_1 có bậc yêu cầu 1 và chỉ kề với c_1 , trực giác tương ứng là:

$$1 \cdot x_{c_1,\text{single}} + 2 \cdot x_{c_1,\text{double}} = 1,$$

các mệnh đề CNF thực tế sẽ được sinh bởi cardinality encoding (kèm biến phụ trợ), do đó tổng số mệnh đề của test case là hữu hạn (ở đây nhóm bỏ qua để tránh dài dòng ví dụ)

2.4.4 Output

SAT solver trả về một mô hình nghiệm thỏa mãn, tương ứng với cấu hình:

$$x_{c_1,\text{single}} = \text{True}, \quad x_{c_2,\text{single}} = \text{True}$$

(tức là mỗi hành lang đặt đúng 1 cầu).

Dưới dạng ma trận ký tự, lời giải được xuất như sau:

$$\begin{bmatrix} 1 & - & 2 \\ 0 & 0 & | \\ 0 & 0 & 1 \end{bmatrix}$$

Trong đó “-” biểu diễn cầu ngang đơn và “|” biểu diễn cầu dọc đơn.

3 Algorithm Implementation

3.1 Input, Output and Checker

Trước khi áp dụng các thuật toán tìm kiếm trạng thái hay suy diễn logic dựa trên SAT, hệ thống cần một cơ chế nhất quán để tiếp nhận dữ liệu đầu vào, biểu diễn bài toán, xuất kết quả và kiểm tra tính hợp lệ của các cấu hình trung gian cũng như nghiệm cuối cùng. Mục này trình bày chi tiết kiến trúc các module liên quan.

3.1.1 Input processing

Việc đọc và phân tích dữ liệu đầu vào được đảm nhiệm bởi module **loader**, với lớp trung tâm là **PuzzleLoader**. Module này có nhiệm vụ chuyển đổi dữ liệu thô từ tập tin văn bản sang cấu trúc dữ liệu lưới nội bộ của hệ thống.

- **Định dạng đầu vào:** Mỗi bài toán được lưu dưới dạng một ma trận số nguyên hai chiều. Các giá trị dương (1–8) biểu thị các đảo và đồng thời là số lượng cầu mà đảo đó yêu cầu, trong khi giá trị 0 biểu thị ô trống (mặt nước). Các phần tử trong cùng một dòng được phân tách bằng dấu phẩy hoặc khoảng trắng.
- **Quy trình phân tích:** **PuzzleLoader** thực hiện đọc từng dòng của tập tin, loại bỏ dòng trống, chuẩn hóa ký tự phân tách và chuyển đổi các token thành số nguyên. Hệ thống kiểm tra tính nhất quán về số cột của mỗi dòng nhằm đảm bảo ma trận đầu vào là hình chữ nhật.

Sau khi kiểm tra hợp lệ, dữ liệu được đóng gói và truyền cho lớp **Grid** để xây dựng biểu diễn bài toán.

3.1.2 Problem representation

Lớp **Grid** chịu trách nhiệm chuyển đổi bài toán từ dạng ma trận hai chiều sang một biểu diễn đồ thị bất biến, phù hợp cho cả thuật toán tìm kiếm và mã hóa SAT.

- **Đỉnh (Island):** Mỗi ô chứa giá trị dương được ánh xạ thành một đối tượng **Island**, bao gồm định danh duy nhất, tọa độ (*row, col*) và số cầu yêu cầu (*target*). Các đảo được lưu trữ trong một bảng ánh xạ để truy xuất nhanh.
- **Cạnh tiềm năng (BridgeCorridor):** Từ mỗi đảo, hệ thống thực hiện quét theo hai hướng ngang và dọc để phát hiện các đảo lân cận có thể nối cầu trực tiếp. Mỗi hành lang hợp lệ giữa hai đảo được biểu diễn bởi một đối tượng **BridgeCorridor**, lưu trữ:
 - hai đảo đầu mút,
 - hướng (ngang hoặc dọc),
 - danh sách các ô lưới nằm giữa hai đảo.

Các hành lang này được tính toán một lần duy nhất khi khởi tạo **Grid** và được xem là tập cạnh tiềm năng cố định của bài toán.

Cách tiếp cận này giúp giảm không gian tìm kiếm, vì các thuật toán chỉ cần quyết định số cầu trên từng hành lang thay vì duyệt toàn bộ lưới ở mỗi bước.

3.1.3 Output and State management

Một cấu hình cụ thể của bài toán tại một thời điểm được biểu diễn bởi lớp **PuzzleState**.

- **Biểu diễn trạng thái:** Thay vì sao chép toàn bộ lưới, trạng thái chỉ lưu trữ một ánh xạ giữa định danh của mỗi **BridgeCorridor** và số lượng cầu hiện có trên hành lang đó (0, 1 hoặc 2). Điều này giúp tối ưu hóa bộ nhớ, đặc biệt quan trọng đối với các thuật toán tìm kiếm sinh nhiều trạng thái trung gian.

- **Thông tin suy diễn:** Từ trạng thái hiện tại, hệ thống có thể tính toán các thuộc tính dẫn xuất như:
 - số cầu còn thiếu tại mỗi đảo,
 - mức độ kết nối của đồ thị,
 - tổng độ thiếu hụt (deficit) dùng cho heuristic.
- **Xuất kết quả (Output):** Để phục vụ hiển thị và kiểm tra nghiệm, trạng thái có thể được chuyển về dạng ma trận ký tự. Các ký hiệu được sử dụng bao gồm:
 - -, = cho cầu ngang đơn và đôi,
 - |, \$ cho cầu dọc đơn và đôi,
 - chữ số cho các đảo.

Dạng biểu diễn này cho phép người dùng và hệ thống đánh giá trực quan lời giải thu được.

3.1.4 Constraint checking

Để đảm bảo tính đúng đắn của các trạng thái trong quá trình tìm kiếm cũng như nghiệm cuối cùng, hệ thống cung cấp module **ConstraintChecker** thực hiện các kiểm tra độc lập.

- **Kiểm tra bậc đảo:** Đảm bảo tổng số cầu nối vào mỗi đảo không vượt quá giá trị yêu cầu và không âm trong các trạng thái trung gian.
- **Kiểm tra giao cắt:** Xác minh không tồn tại hai cầu vuông góc cắt nhau tại cùng một ô lưới, một ràng buộc quan trọng của bài toán Hashiwokakero.
- **Kiểm thử nghiệm:** Một trạng thái được xem là nghiệm hợp lệ nếu và chỉ nếu thỏa mãn đồng thời:
 1. Tất cả các đảo đều đạt đúng số cầu yêu cầu
 2. Đồ thị cầu là liên thông
 3. Không vi phạm bất kỳ ràng buộc hình học nào

3.2 CNF Generation and PySAT Solver

Để giải bài toán Hashiwokakero bằng logic mệnh đề, hệ thống được xây dựng dựa trên thư viện **python-sat** (PySAT) – một framework mạnh mẽ cho phép tương tác trực tiếp với nhiều bộ giải SAT hiện đại như Glucose, Minisat hay Cadical. Kiến trúc tổng thể được thiết kế theo hai giai đoạn chính: *mã hoá ràng buộc (CNF Encoding)* và *giải lặp có phản hồi (Iterative Solving with Lazy Constraints)*.

3.2.1 Automated CNF Encoding

Quá trình chuyển đổi bài toán từ biểu diễn đồ thị sang công thức logic chuẩn hội (CNF) được đóng gói trong lớp **CNFEncoder**. Lớp này hoạt động độc lập với bộ giải, cho phép dễ dàng kiểm thử và thay thế chiến lược mã hoá khi cần thiết.

Quản lý biến logic. SAT solver yêu cầu mỗi biến logic phải được ánh xạ sang một số nguyên dương duy nhất theo chuẩn DIMACS. Hệ thống sử dụng lớp `VariableRegistry` để đảm nhiệm vai trò này. Mỗi biến ngữ nghĩa (ví dụ: `corridor_single` của hành lang c) được ánh xạ nhất quán sang một ID số nguyên.

Cơ chế ánh xạ này có hai vai trò quan trọng:

- Đảm bảo không xảy ra xung đột ID khi sinh thêm biến phụ trợ trong các ràng buộc đếm
- Cho phép suy diễn ngược (decoding) từ mô hình nghiệm SAT về trạng thái bài toán ban đầu một cách chính xác

Sinh các nhóm ràng buộc. Lớp `CNFEncoder` duyệt qua cấu trúc đồ thị `Grid` và sinh các mệnh đề CNF tương ứng với từng nhóm ràng buộc đã được xác định trong mô hình lý thuyết:

- **Ràng buộc miền giá trị (Domain Constraints):** Đảm bảo một hành lang không thể đồng thời chứa cầu đơn và cầu đôi, đồng thời liên kết biến phụ trợ **active** với hai biến trạng thái này.
- **Ràng buộc giao cắt (Crossing Constraints):** Dựa trên danh sách các ô lưới mà mỗi hành lang chiếm dụng, hệ thống tự động phát hiện các cặp hành lang ngang–dọc giao nhau. Với mỗi giao điểm, một mệnh đề phủ định đồng thời (*NAND clause*) được sinh ra để ngăn hai hành lang cùng được kích hoạt.
- **Ràng buộc bậc đảo (Weighted Degree Constraints):** Với mỗi đảo có giá trị mục tiêu k , phương trình

$$\sum (1 \cdot x_{\text{single}} + 2 \cdot x_{\text{double}}) = k$$

được hiện thực hóa bằng cách kết hợp hai ràng buộc *AtLeast* và *AtMost*. Để xử lý trọng số, mỗi biến cầu đôi được quy đổi thành hai đơn vị tương đương trong danh sách các biến đếm. Module `CardEnc` của PySAT được sử dụng để sinh các mệnh đề CNF tương ứng một cách tự động và tối ưu.

Kết quả của giai đoạn này là một tập mệnh đề CNF hoàn chỉnh, mô tả đầy đủ các ràng buộc cục bộ của bài toán.

3.2.2 PySAT Solver & Lazy Constraints

Lớp `PySatSolver` đóng vai trò là động cơ suy diễn trung tâm. Thay vì mã hoá toàn bộ ràng buộc liên thông ngay từ đầu (dẫn đến số lượng mệnh đề rất lớn), hệ thống áp dụng chiến lược **Lazy Constraint Generation**, trong đó các ràng buộc toàn cục chỉ được thêm vào khi thật sự cần thiết.

Vòng lặp giải có phản hồi. Quy trình thực thi của bộ giải được mô tả như sau:

1. **Khởi tạo:** Khởi tạo một đối tượng SAT solver và nạp tập mệnh đề CNF cơ bản (chưa bao gồm ràng buộc liên thông).

2. **Giải SAT:** Gọi `solver.solve()` để tìm một mô hình nghiệm M thỏa mãn các ràng buộc hiện tại. Nếu không tồn tại nghiệm, kết luận bài toán vô nghiệm.
3. **Suy diễn trạng thái:** Nếu tìm được M , mô hình nghiệm được chuyển đổi thành đối tượng `PuzzleState` thông qua lớp `InferenceEngine`. Trạng thái này biểu diễn một cấu hình cầu cụ thể trên lưới.
4. **Kiểm tra liên thông:** Hệ thống sử dụng thuật toán **Disjoint-Set Union (Union-Find)** để xác định các thành phần liên thông của đồ thị cầu. Mỗi hành lang được kích hoạt sẽ hợp nhất hai đảo đầu mút của nó.
5. **Sinh mệnh đề cắt:** Nếu đồ thị bị phân mảnh thành nhiều thành phần, với mỗi thành phần không chứa đảo gốc, hệ thống xác định tập các hành lang tiềm năng có thể nối thành phần đó với phần còn lại. Một mệnh đề cắt dạng:

$$\bigvee x_{c,\text{active}}$$

được sinh ra và nạp ngược lại vào solver thông qua `solver.add_clause()`.

Hội tụ và tính hiệu quả. Quá trình trên được lặp lại cho đến khi:

- Tìm được một nghiệm SAT mà đồ thị cầu liên thông hoàn toàn, hoặc
- Công thức CNF trở nên không thỏa mãn (UNSAT).

3.3 A* Search Algorithm

Việc giải bài toán Hashiwokakero bằng thuật toán A* đòi hỏi phải xây dựng các hàm đánh giá tuân thủ chặt chẽ nguyên tắc cận dưới. Hiệu quả của A* khi giải bài toán này phụ thuộc mật thiết với độ chặt chẽ của hàm heuristic $h(n)$, tức khả năng của nó trong việc xấp xỉ chi phí còn lại thực sự $h^*(n)$ mà không được phép đánh giá vượt quá. Trong bối cảnh trò chơi nối cầu này, các heuristic chủ yếu được xây dựng dựa trên việc relax các ràng buộc, cụ thể các ràng buộc về sự kết nối, thành phần liên thông, trong khi duy trì đầy đủ các ràng buộc của bậc các nút và cấu trúc của đồ thị.

Phân tích cho thấy các bộ giải mạnh thường dùng một hệ thống nhiều tầng của các loại heuristic. Chúng đi từ những cách đếm đơn giản ở mức cục bộ, vốn rất rẻ để tính, cho đến những cận dưới phức tạp hơn được rút ra từ bài toán quy hoạch tuyến tính rời rạc hoặc từ các kỹ thuật cắt luồng trong mạng. Nhìn chung, các heuristic này rơi vào ba nhóm: loại dựa trên lượng bậc còn thiếu của từng nút, loại đánh giá mức độ liên thông của toàn bộ đồ thị, và loại dùng các phương pháp tối ưu hóa như LP hoặc Max-Flow để tạo ra cận dưới tốt hơn. Trong báo cáo lần này, chúng em đề xuất các heuristic sau:

- **Deficit Heuristic:** Xuất phát từ việc tính toán số bậc còn lại tại mỗi đỉnh để ước lượng số cạnh tối thiểu cần thêm vào.
- **Connectivity Component Heuristic:** Ước lượng số cạnh tối thiểu cần thêm vào dựa trên việc relax bài toán về các nhóm đồ thị liên thông.
- **Local Bottleneck Heuristic:** Đánh giá các ràng buộc cục bộ khi bậc yêu cầu của đảo lớn hơn số hướng đi khả dụng, dẫn đến việc buộc phải sử dụng cầu đôi.

- **Composit Heuristic:** là max của các heuristic nêu trên, để đạt được một cận dưới gần với h^* nhất

Báo cáo sẽ chứng minh tính admissible và consistent của các heuristic.

Định nghĩa 1 (Admissible). Một hàm heuristic $h(n)$ được gọi là *admissible* nếu với mọi nút n trong không gian trạng thái, ta có:

$$0 \leq h(n) \leq h^*(n) \quad (1)$$

Trong đó $h^*(n)$ là chi phí tối ưu thực tế để đi từ n đến trạng thái đích. Tính chất này đảm bảo thuật toán A^* luôn tìm được giải pháp tối ưu (nếu tồn tại) khi sử dụng tìm kiếm trên cây (tree search).

Định nghĩa 2 (Consistent). Một hàm heuristic $h(n)$ được gọi là *consistent* nếu với mọi nút n và nút kế tiếp n' được tạo ra bởi hành động có chi phí $c(n, n')$, bất đẳng thức sau luôn thỏa mãn:

$$h(n) \leq c(n, n') + h(n') \quad (2)$$

Trong tìm kiếm trên đồ thị (graph search), tính chất consistent đảm bảo rằng khi một nút được mở rộng, đường đi tìm được đến đó chính là đường đi tối ưu nhất, giúp loại bỏ việc phải xử lý lại các nút đã đóng.

3.3.1 Deficit Heuristic

Deficit Heuristic là heuristic cơ bản và trực quan nhất cho bài toán Hashiwokakero, dựa trên nguyên lý cơ bản của lý thuyết đồ thị: Định lý bắt tay (Handshaking Lemma). Ý tưởng cốt lõi của heuristic này là bỏ qua các ràng buộc về hình học (vị trí, sự cắt nhau của các cạnh) và tính liên thông, chỉ tập trung duy nhất vào việc thỏa mãn "cơn khát" về bậc (degree) của các hòn đảo. Nó giả định một kịch bản lý tưởng nơi mọi nhu cầu về bậc đều có thể được đáp ứng ngay lập tức mà không bị cản trở.

Gọi V là tập hợp các hòn đảo trong đồ thị. Tại trạng thái n , với mỗi đảo $v \in V$, gọi r_v là bậc còn lại cần phải thỏa mãn. Với $r_v = \text{bậc mục tiêu}_v - \text{bậc hiện tại}_v$, gọi tổng bậc dư của toàn bộ đồ thị tại trạng thái n , Heuristic phát biểu như sau:

$$h_{\text{deficit}} = \frac{R(n)}{2} = \frac{\sum_{v \in V} r_v}{2}$$

Chứng minh tính Admissible: Để chứng minh $h_{\text{deficit}}(n)$ là admissible, ta cần chứng minh $h_{\text{deficit}}(n) \leq h^*(n)$ với mọi trạng thái n , trong đó $h^*(n)$ là số lượng cầu tối thiểu thực tế cần thêm vào để đạt được trạng thái đích.

Xét một hành động a bất kỳ (thêm một cây cầu nối hai đảo u và v). Hành động này làm giảm bậc còn lại của u đi 1 và v đi 1. Do đó, tổng bậc dư của đồ thị giảm đi đúng 2 đơn vị sau mỗi bước đi: $R(n') = R(n) - 2$

Để đạt được trạng thái đích (goal state), tất cả các đảo phải có $r_v = 0$, tức là $R(\text{goal}) = 0$. Vì mỗi bước đi chỉ có thể giảm tổng R đi tối đa là 2, số bước đi tối thiểu cần thiết k để đưa $R(n)$ về 0 được xác định bởi:

$$R(n) - 2k = 0 \implies k = \frac{R(n)}{2}$$

Giá trị này chính là công thức của heuristic:

$$h_{deficit}(n) = \frac{\sum_{v \in V} r_v}{2}$$

Trong thực tế, do các ràng buộc về việc chặn đường hoặc tính liên thông, số lượng cầu cần thiết có thể lớn hơn hoặc bài toán có thể không giải được, nhưng không bao giờ có thể nhỏ hơn con số lý thuyết này. Do đó:

$$h_{deficit}(n) \leq h^*(n)$$

Vậy, Deficit Heuristic là admissible.

Chứng minh tính Consistent Khi thực hiện hành động thêm cầu nối u và v , bậc còn lại (residual degree) của đảo u giảm đi 1 và của đảo v giảm đi 1. Tổng bậc dư của toàn bộ đồ thị thay đổi như sau:

$$R(n') = R(n) - 1 - 1 = R(n) - 2$$

Tính toán giá trị heuristic tại trạng thái mới n' :

$$\begin{aligned} h_{deficit}(n') &= \frac{R(n')}{2} = \frac{R(n) - 2}{2} = \frac{R(n)}{2} - 1 \\ \implies h_{deficit}(n') &= h_{deficit}(n) - 1 \end{aligned}$$

Thay thế vào bất đẳng thức 2

$$\begin{aligned} h_{deficit}(n) &\leq 1 + h_{deficit}(n') \\ \iff h_{deficit}(n) &\leq 1 + (h_{deficit}(n) - 1) \\ \iff h_{deficit}(n) &\leq h_{deficit}(n) \end{aligned}$$

Biểu thức trên luôn đúng (thực tế là dấu bằng xảy ra). Điều này chứng tỏ $h_{deficit}$ giảm đi chính xác bằng chi phí của bước đi mỗi khi một cây cầu được thêm vào.

Kết luận: $h_{deficit}$ thỏa mãn tính chất Consistent.

3.3.2 Connectivity Component Heuristic

Trong khi Deficit Heuristic tập trung vào các ràng buộc cục bộ tại mỗi đỉnh, nó có một nhược điểm lớn: nó không xem được trạng thái liên thông của toàn đồ thị. Một trạng thái có thể thỏa mãn toàn bộ các yêu cầu về bậc ($h_{deficit} = 0$) nhưng vẫn chưa phải là trạng thái đích nếu nó bao gồm hai hoặc nhiều cụm tách biệt không có đường nối. Connectivity Component Heuristic được xây dựng để giải quyết vấn đề này bằng cách xem xét cấu trúc toàn cục của đồ thị.

Heuristic này relax bài toán Hashiwokakero về bài toán tìm cây khung nhỏ nhất (Minimum Spanning Tree - MST) trên đồ thị tổng quát. Gọi $k(n)$ là số lượng thành phần liên thông hiện tại của đồ thị tại trạng thái n . Để tất cả các đảo nối lại thành một mạng lưới duy nhất, ta cần giảm số lượng thành phần liên thông từ $k(n)$ xuống còn 1.

Công thức của heuristic được định nghĩa như sau:

$$h_{conn}(n) = k(n) - 1$$

Chứng minh tính Admissible: Theo lý thuyết đồ thị, một đồ thị vô hướng với k thành phần liên thông cần thêm tối thiểu $k - 1$ cạnh để trở nên liên thông (tạo thành một cây khung nối các thành phần lại với nhau).

Xét hành động a thêm một cây cầu nối hai đảo u và v bất kỳ:

- **Trường hợp 1:** Nếu u và v thuộc cùng một thành phần liên thông, số lượng thành phần liên thông không đổi ($k' = k$).
- **Trường hợp 2:** Nếu u và v thuộc hai thành phần liên thông khác nhau, hai thành phần này sẽ hợp nhất làm một. Số lượng thành phần liên thông giảm đi 1 ($k' = k - 1$).

Như vậy, mỗi bước đi (chi phí = 1) chỉ có thể làm giảm giá trị $k(n)$ tối đa là 1 đơn vị. Để đi từ trạng thái hiện tại có $k(n)$ thành phần đến trạng thái đích (liên thông hoàn toàn) có 1 thành phần, số bước đi tối thiểu m phải thỏa mãn:

$$k(n) - 1 \cdot m \leq 1 \implies m \geq k(n) - 1$$

Do đó, chi phí thực tế $h^*(n)$ luôn lớn hơn hoặc bằng $k(n) - 1$. Trong bài toán Hashiwokakero thực tế, do các ràng buộc về crossing edges và ràng buộc về bậc, việc nối các thành phần liên thông có thể tốn nhiều hơn 1 cầu, nhưng không bao giờ tốn ít hơn con số lý thuyết này.

$$h_{conn}(n) = k(n) - 1 \leq h^*(n)$$

Kết luận: Connectivity Component Heuristic là admissible.

Chứng minh tính Consistent: Xét hành động chuyển từ trạng thái n sang n' bằng việc thêm một cầu với chi phí $c(n, n') = 1$. Bất đẳng thức consistency $h_{conn}(n) \leq 1 + h_{conn}(n')$ tương đương với $k(n) - 1 \leq 1 + (k(n') - 1) \iff k(n) \leq k(n') + 1$. Khi thêm một cạnh vào đồ thị, số lượng thành phần liên thông chỉ có thể thay đổi theo hai hướng: hoặc giảm đi 1 nếu cạnh nối hai thành phần rời rạc ($k(n') = k(n) - 1$), hoặc giữ nguyên nếu cạnh tạo chu trình trong cùng một thành phần ($k(n') = k(n)$). Thay thế vào điều kiện trên, ta thấy bất đẳng thức luôn thỏa mãn (dấu bằng xảy ra ở trường hợp hợp nhất, và nhỏ hơn hẳn ở trường hợp tạo chu trình). Do đó, h_{conn} thỏa mãn tính chất Consistent.

3.3.3 Local Bottleneck Heuristic

Trong khi Deficit Heuristic giả định mọi hướng đi đều khả dụng, Local Bottleneck Heuristic xem xét các giới hạn về không gian tại từng đảo cụ thể. Heuristic này dựa trên quan sát: nếu một hòn đảo có bậc còn thiếu r_v nhưng chỉ có a_v hướng đi khả dụng và hợp lệ, và $r_v > a_v$, thì theo nguyên lý chuồng bồ câu (Pigeonhole Principle), ta buộc phải đặt thêm ít nhất $r_v - a_v$ cây cầu kép lên các hướng đi đó.

Công thức được xác định như sau:

$$h_{bot}(n) = \left\lceil \frac{1}{2} \sum_{v \in V} \max(0, r_v - a_v) \right\rceil$$

Trong đó a_v là số lượng các cạnh lân cận của đảo v chưa bão hòa (có thể nhận thêm cầu).

Chứng minh tính Admissible: Ta chứng minh $h_{bot}(n) \leq h^*(n)$ thông qua tính chất bắc cầu với Deficit Heuristic. Xét biểu thức thành phần của heuristic tại một đảo v bất kỳ: $\max(0, r_v - a_v)$. Vì số lượng hướng đi khả dụng a_v luôn là một số không âm ($a_v \geq 0$), ta luôn có bất đẳng thức:

$$r_v - a_v \leq r_v$$

Lấy tổng hai vế trên toàn bộ tập đỉnh V và chia cho 2:

$$\begin{aligned} \frac{1}{2} \sum_{v \in V} (r_v - 2 \cdot a_v) &\leq \frac{1}{2} \sum_{v \in V} r_v \\ \implies h_{bot}(n) &\leq h_{deficit}(n) \end{aligned}$$

Ở phần trước, ta đã chứng minh $h_{deficit}(n) \leq h^*(n)$. Do tính chất bắc cầu:

$$h_{bot}(n) \leq h_{deficit}(n) \leq h^*(n)$$

Như vậy, Local Bottleneck Heuristic luôn đưa ra một ước lượng nhỏ hơn hoặc bằng chi phí thực tế, đảm bảo tính Admissible. Mặc dù về mặt giá trị đại số, heuristic này thường nhỏ hơn Deficit, nhưng giá trị thực tiễn của nó nằm ở việc phát hiện các trạng thái không thể tìm ra tiếp lời giải (khi $r_v > a_v$, tức nhu cầu vượt quá khả năng cung ứng tối đa), giúp cắt tỉa nhánh sớm hơn là cung cấp một cận dưới số học chặt hơn.

Chứng minh tính Consistent: Xét sự thay đổi của đại lượng $D_v = r_v - a_v$ khi thêm một cầu nối giữa u và v . Tại mỗi đầu cầu (ví dụ u), bậc yêu cầu r_u luôn giảm 1. Số hướng đi khả dụng a_u có thể giữ nguyên (nếu cạnh vừa thêm là cầu thứ nhất) hoặc giảm 1 (nếu cạnh vừa thêm là cầu thứ hai gây bão hòa hướng đó). Do đó, giá trị D_u sẽ giảm 1 (nếu a_u giữ nguyên) hoặc không đổi (nếu a_u giảm).

Đặt $S(n) = \sum_{v \in V} \max(0, r_v - a_v)$. Từ phân tích ở trên, ta thấy tại mỗi đầu cầu u và v , giá trị D giảm tối đa 1 đơn vị. Do đó, tổng S tại trạng thái mới n' thỏa mãn:

$$S(n') \geq S(n) - 1 - 1 = S(n) - 2$$

Xét hiệu số giữa giá trị heuristic tại hai trạng thái:

$$h_{bot}(n) - h_{bot}(n') = \left\lceil \frac{S(n)}{2} \right\rceil - \left\lceil \frac{S(n')}{2} \right\rceil$$

Sử dụng tính chất của hàm trần $\lceil x - k \rceil = \lceil x \rceil - k$ (với k là số nguyên) và bất đẳng thức $S(n') \geq S(n) - 2$:

$$\left\lceil \frac{S(n')}{2} \right\rceil \geq \left\lceil \frac{S(n) - 2}{2} \right\rceil = \left\lceil \frac{S(n)}{2} - 1 \right\rceil = \left\lceil \frac{S(n)}{2} \right\rceil - 1$$

Suy ra:

$$\begin{aligned} h_{bot}(n') &\geq h_{bot}(n) - 1 \\ \iff h_{bot}(n) &\leq 1 + h_{bot}(n') \end{aligned}$$

Vì chi phí thực tế của bước đi là $c(n, n') = 1$, bất đẳng thức trên hoàn toàn tương đương với điều kiện consistency:

$$h(n) \leq c(n, n') + h(n')$$

3.3.4 Composite Heuristic

Composite Heuristic được định nghĩa là giá trị lớn nhất của các heuristic thành phần tại mỗi trạng thái:

$$h_{comp}(n) = \max\{h_{deficit}(n), h_{conn}(n), h_{bot}(n)\}$$

Tính Admissible và Consistent: Do tất cả các heuristic thành phần ($h_{deficit}$, h_{conn} , h_{bot}) đều đã được chứng minh là admissible và consistent ở các phần trước, heuristic tổng hợp h_{comp} kế thừa hoàn toàn các tính chất này dựa trên các bất đẳng thức đại số:

- **Admissible:** Vì $h_i(n) \leq h^*(n)$ với mọi thành phần i , suy ra $\max\{h_i(n)\} \leq h^*(n)$.
- **Consistent:** Gọi h_k là heuristic đạt giá trị cực đại tại trạng thái n . Ta có chuỗi bất đẳng thức:

$$h_{comp}(n) = h_k(n) \leq 1 + h_k(n') \leq 1 + h_{comp}(n')$$

Kết luận, h_{comp} thỏa mãn cả hai tính chất cần chứng minh

3.3.5 Forward checking

Để tối ưu hóa quá trình tìm kiếm của thuật toán A*, nhóm tích hợp thêm cơ chế Forward Checking[1] nhằm dự đoán sớm các trạng thái không khả thi. Thay vì chỉ kiểm tra tính hợp lệ của các cầu đã đặt, FC thực hiện đánh giá khả năng hoàn thành các đảo trong tương lai.

Với mỗi đảo i trong trạng thái hiện tại, thuật toán thực hiện các bước kiểm tra sau:

- **Tính toán số cầu còn thiếu:** Gọi $target(i)$ là số bậc yêu cầu của đảo và $v_{current}(c)$ là số lượng cầu hiện có trên hành lang c . Số cầu còn thiếu (remaining) được xác định bởi:

$$rem(i) = target(i) - \sum_{c \in \mathcal{C}(i)} v_{current}(c)$$

Trong đó $\mathcal{C}(i)$ là tập hợp các hành lang kết nối với đảo i . Nếu $rem(i) < 0$, trạng thái vi phạm ràng buộc và bị loại bỏ ngay lập tức.

- **Xác định các hành lang khả thi:** Một hành lang c được coi là khả thi để thêm cầu nếu nó thỏa mãn đồng thời:

1. Số cầu hiện tại $v_{current}(c) < 2$.
2. Không bị chặn bởi một hành lang khác đã có cầu cắt ngang qua (Crossing check).

Gọi $\mathcal{C}_{feasible}(i)$ là tập các hành lang thỏa mãn điều kiện trên.

- **Đánh giá khả năng đáp ứng (Conservative Pruning):** Tính tổng số cầu tối đa có thể bổ sung thêm cho đảo i :

$$max_add(i) = \sum_{c \in \mathcal{C}_{feasible}(i)} (2 - v_{current}(c))$$

Nếu $max_add(i) < rem(i)$, điều đó có nghĩa là ngay cả khi tất cả các hành lang tiềm năng đều được khai thác tối đa, đảo i vẫn không bao giờ đạt được bậc mục tiêu. Khi đó, nhánh tìm kiếm này sẽ bị cắt tỉa (prune).

Cơ chế này giúp A^* không lãng phí chi phí tính toán để mở rộng các trạng thái mà về mặt toán học chắc chắn không thể dẫn đến lời giải, từ đó thu hẹp đáng kể không gian trạng thái cần duyệt.

3.3.6 Complexity Analysis

Hiệu quả của thuật toán A^* phụ thuộc chặt chẽ vào cấu trúc không gian trạng thái và khả năng định hướng của hàm heuristic. Để phân tích độ phức tạp, ta định nghĩa các tham số kích thước của bài toán Hashiwokakero:

- $|\mathcal{V}|$: Số lượng đảo (Islands).
- $|\mathcal{C}|$: Số lượng hành lang khả dĩ (Potential bridges/corridors).

1. Space Complexity

Độ phức tạp không gian được xác định bởi dung lượng bộ nhớ cần thiết để lưu trữ hai tập: *Open Set* và *Closed Set*.

- *Số lượng trạng thái tối đa*: Mỗi hành lang $c \in \mathcal{C}$ có thể nhận một trong 3 giá trị cầu hình: không cầu, cầu đơn, hoặc cầu đôi ($v \in \{0, 1, 2\}$). Do đó, kích thước tối đa của không gian trạng thái là $3^{|\mathcal{C}|}$.
- *Kích thước mỗi trạng thái*: Tại mỗi nút, hệ thống cần lưu trữ ánh xạ giá trị của toàn bộ danh sách hành lang, tương ứng với kích thước $O(|\mathcal{C}|)$.

Trong trường hợp xấu nhất, khi thuật toán phải lưu trữ phần lớn không gian trạng thái, độ phức tạp không gian là:

$$Space(A^*) = O(|\mathcal{C}| \cdot 3^{|\mathcal{C}|})$$

2. Time Complexity (Độ phức tạp thời gian)

Độ phức tạp thời gian được tính bằng tích giữa số lượng nút được mở rộng ($N_{expanded}$) và chi phí tính toán tại mỗi nút (T_{node}). Chi phí tính toán T_{node} bị chi phối bởi hàm heuristic phức tạp nhất là h_{conn} , đòi hỏi duyệt đồ thị với chi phí $O(|\mathcal{V}| + |\mathcal{C}|)$.

Ta xem xét hai trường hợp cho $N_{expanded}$:

- *Trường hợp xấu nhất*: Nếu heuristic không hiệu quả (hoặc $h(n) = 0$), thuật toán không thể cắt tỉa nhánh và phải duyệt toàn bộ không gian. Hệ số nhánh (b) đạt tối đa bằng 3.

$$T_{worst} = O((|\mathcal{V}| + |\mathcal{C}|) \cdot 3^{|\mathcal{C}|})$$

- *Trường hợp hiệu dụng*: Với các heuristic mạnh (như Composite Heuristic), không gian tìm kiếm được cắt tỉa đáng kể. Gọi b^* là **hệ số nhánh hiệu dụng**, với $1 \leq b^* \ll 3$. Độ phức tạp thực tế giảm xuống:

$$T_{effective} = O((|\mathcal{V}| + |\mathcal{C}|) \cdot (b^*)^{|\mathcal{C}|})$$

3.4 Baseline Algorithms

3.4.1 Brute force

Để giải bài toán này với cách tiếp cận thông thường, nhóm xây dựng 1 thuật toán brute force. Với ý tưởng duyệt toàn bộ không gian nghiệm có thể có của lời giải.

Ý tưởng tổng quát: Mỗi hành lang nối giữa 2 đảo được gán một trong ba giá trị: $v \in \{0, 1, 2\}$ tương ứng với không đặt cầu, đặt 1 cầu hoặc đặt 2 cầu. Thuật toán sẽ duyệt qua tất cả các tổ hợp bằng thuật toán tìm kiếm theo chiều sâu (DFS).

Sắp xếp thứ tự theo MRV Để cải thiện tốc độ duyệt, nhóm sử dụng heuristic *Minimum Remaining Values* (MRV) khi lựa chọn thứ tự các hành lang cần được gán. Cụ thể ta sẽ ưu tiên xử lý các hành lang nối giữa 2 đảo có min giá trị nhỏ nhất, nếu bằng thì xét đến tổng 2 giá trị (ưu tiên nhỏ hơn). Vì đây là nhóm có ràng buộc chặt hơn và việc gán sớm giúp giảm đáng kể chi phí tính toán về sau.

Điều kiện nghiệm Khi tất cả hành lang đều được gán giá trị, trạng thái được xem là nghiệm hợp lệ nếu thỏa:

- Tất cả các đảo đều đạt đúng tổng số cầu theo mục tiêu đặt ra;
- Toàn bộ cấu trúc cầu tạo thành một đồ thị liên thông.

3.4.2 Backtracking

Ngoài cách Brute Force, vốn tiêu tốn nhiều tài nguyên do phải duyệt toàn bộ các nghiệm có thể có trong không gian nghiệm khổng lồ. Thuật toán Backtracking chủ động loại bỏ sớm những nhánh dẫn đến trạng thái không hợp lệ, nhờ đó giảm đáng kể số lượng trạng thái cần duyệt.

Ý tưởng cốt lõi Thuật toán tiến hành gán giá trị cho các hành lang theo từng bước. Sau mỗi lần gán, trạng thái hiện tại sẽ được check các ràng buộc cục bộ (ví dụ như số cầu vượt quá mục tiêu, hình dạng cầu hợp lệ không, ..), thuật toán sẽ quay lui và thử giá trị khác.

Heuristic MRV cho đảo Nhóm thử 1 Heuristic khác với bruteforce xét theo hành lang, thuật toán backtracking của nhóm sử dụng heuristic MRV dựa trên *remaining degree* của đảo:

$$\text{remaining degree}(i) = \text{target}(i) - \text{current degree}(i)$$

Thuật toán luôn chọn đảo có giá trị remaining nhỏ nhất để xử lý trước. Vì đây là những đảo có ràng buộc chặt nhất.

Điều kiện dừng Thuật toán trả về nghiệm khi tất cả các đảo đều đạt đúng target và không còn dư cầu cần gán và thỏa được điều kiện liên thông. Nếu không có lựa chọn hợp lệ nào còn lại, toàn bộ nhánh được loại bỏ.

Forward Checking Để tiến thêm một bước trong tối ưu, nhóm bổ sung cơ chế Forward Checking [1], một kỹ thuật mạnh trong các bài toán ràng buộc. Khi gán giá trị v cho một hành lang c , FC cập nhật lại domain của các hành lang lân cận theo các quy tắc:

- Nếu một đảo i đã thoả $\text{deg}_{\text{current}}(i) = \text{target}(i)$, mọi hành lang còn lại nối vào i chỉ được phép nhận các giá trị không vượt quá giá trị hiện tại.
- Với mỗi đảo i , tính giá trị cực đại có thể đạt được:

$$\text{max_possible}(i) = \sum_{c \in \mathcal{C}(i)} \max\{v_{\text{current}}(c), \max(D_c)\},$$

trong đó D_c là domain hiện tại của hành lang c . Nếu $\text{max_possible}(i) < \text{target}(i)$, trạng thái hiện tại chắc chắn không dẫn đến nghiệm.

- Nếu domain của bất kỳ hành lang nào trở nên rỗng ($|D_c| = 0$), nhánh được loại bỏ ngay.

Nhờ Forward Checking, thuật toán không chỉ kiểm tra tính hợp lệ của trạng thái hiện tại mà còn đánh giá được khả năng tồn tại của nghiệm trong các bước tiếp theo. Điều này giúp cắt bỏ nhiều nhánh sai ngay từ tầng nông nhất của cây tìm kiếm, giảm đáng kể chi phí tính toán.

4 Experiments and Analysis

4.1 Experimental Setup

4.1.1 Test case

Nhằm đảm bảo tính khả thi và độ tin cậy của dữ liệu thử nghiệm, nghiên cứu này áp dụng phương pháp sinh bài toán từ lời giải. Thay vì khởi tạo các vị trí đảo ngẫu nhiên rồi kiểm tra điều kiện tồn tại lời giải, quy trình này thực hiện xây dựng một cấu hình các cầu nối hợp lệ trước, sau đó trích xuất các giá trị số tại mỗi đảo. Cách tiếp cận này đảm bảo mọi bộ dữ liệu tạo ra luôn tồn tại ít nhất một lời giải và đồ thị kết quả luôn là một đồ thị liên thông.

Cơ chế hoạt động: Quy trình xây dựng được lấy cảm hứng từ thuật toán cây khung Prim [2]. Thuật toán bắt đầu bằng việc chọn một vị trí gốc ngẫu nhiên, sau đó mở rộng tập đỉnh thông qua các cạnh có trọng số $w \in \{1, 2\}$. Sau khi hình thành được một cây khung cơ bản, thuật toán thực hiện tăng độ phức tạp của cây khung bằng cách duyệt các cặp đảo thẳng hàng chưa có kết nối và thêm cạnh dựa trên xác suất p cho trước. Điều này làm tăng độ phức tạp cho bài toán nhờ việc tạo ra các chu trình và đa cạnh.

Thuật toán gồm các bước chính:

Algorithm 1 Bridges Puzzle Generation

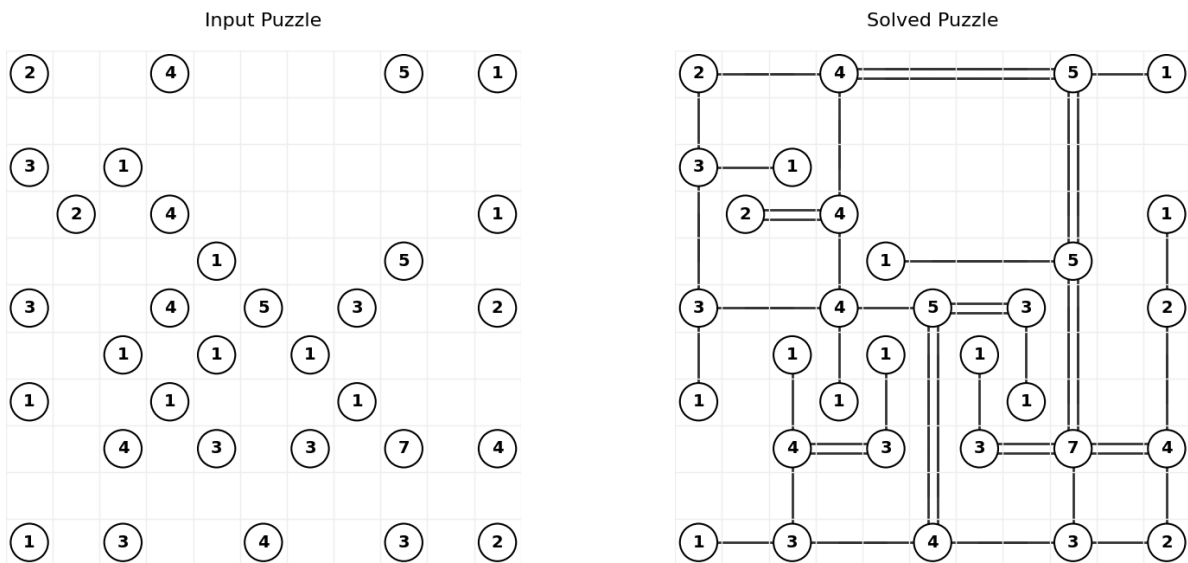
```

1: Input: Grid size  $S$ , Target islands  $N$ , Probabilities  $p, q$ 
2: Output: A valid puzzle (island positions and their degrees)
3:  $I \leftarrow \{i_0\}$  ▷ Khởi tạo tập hợp đảo với một vị trí ngẫu nhiên
4: while  $|I| < N$  do
5:   Chọn ngẫu nhiên  $i \in I$  và hướng Manhattan  $d \in \{N, S, E, W\}$ 
6:   Xác định đảo mới  $j$  theo hướng  $d$  từ  $i$ 
7:   if  $j$  hợp lệ (nằm trong  $S$ , không chồng lấp, không cắt cạnh) then
8:      $I \leftarrow I \cup \{j\}$ 
9:     Kết nối  $(i, j)$  với trọng số  $w \in \{1, 2\}$  dựa trên xác suất  $q$ 
10:    Đánh dấu đường đi giữa  $(i, j)$  là occupied
11:   end if
12:   if không thể mở rộng sau  $K$  lần thử then Restart thuật toán
13: end while
14: for mọi cặp đảo  $(u, v) \in I$  thẳng hàng và chưa kết nối do
15:   if đường nối  $(u, v)$  không gây cắt cạnh then
16:     Thêm cạnh  $(u, v)$  với xác suất  $p$ 
17:   end if
18: end for
19: return Đảo  $i \in I$  với giá trị là tổng trọng số các cạnh kết nối

```

Trong quá trình thực hiện, do các ràng buộc về không gian lưới và quy tắc không giao cắt của các cạnh, thuật toán có xác suất không thể tạo ra đúng số lượng đỉnh. Để đảm bảo tính nhất quán của dữ liệu thử nghiệm, nhóm áp dụng chiến lược thực hiện lặp lại nhiều lần cho đến khi cấu hình bài toán thỏa mãn hoàn toàn các thông số đầu vào.

Hình 4.1 minh họa một test case được khởi tạo ngẫu nhiên trên lưới kích thước 15x15 thông qua thuật toán đã mô tả.



Hình 4.1: Ví dụ về một bộ test case trên lưới 15x15.

4.1.2 Difficulty Configurations

Các bài toán được phân loại theo kích thước lưới và mật độ đảo. Bảng 4.1 chi tiết các thông số thiết lập cho từng mức độ khó:

Bảng 4.1: Thông số cấu hình chi tiết cho các bộ test case

Kích thước lưới	Số lượng đảo (min – max)	Khoảng cách tối thiểu	Xác suất cầu đôi (P_q)	Xác suất cầu phụ (P_p)
7x7	10 – 15	2	0.3	0.4
9x9	25 – 30	2	0.3	0.4
11x11	30 – 35	2	0.3	0.4
13x13	30 – 35	2	0.5	0.4
17x17	35 – 40	2	0.5	0.4

4.2 Performance Metrics

Để đánh giá và so sánh hiệu quả của các thuật toán, nhóm sử dụng các thước đo định lượng sau, được thu thập tự động thông qua module `benchmark.py` và trực quan hóa bằng `visualize_plots.py`:

- **Thời gian thực thi:**

- **Định nghĩa:** Tổng thời gian CPU (wall-clock time) từ lúc thuật toán bắt đầu chạy cho đến khi tìm ra lời giải hoặc kết luận vô nghiệm.
- **Phương pháp đo:** Sử dụng hàm `time.perf_counter()` trong Python để đảm bảo độ chính xác cao.

- **Xử lý dữ liệu:** Trên các biểu đồ, giá trị hiển thị là **thời gian trung bình** của tất cả các test case có cùng kích thước (số đảo hoặc kích thước lưới).
- **Số nút đã duyệt:**
 - **Định nghĩa:** Số lượng trạng thái mà thuật toán đã lấy ra khỏi hàng đợi ưu tiên (nếu là A^*) hoặc số lần gọi đệ quy (nếu là Backtracking).
 - **Ý nghĩa:** Phản ánh khối lượng công việc logic thực sự, độc lập với phần cứng.
- **Bộ nhớ sử dụng:**
 - **Định nghĩa:** Lượng bộ nhớ RAM tối đa mà thuật toán chiếm dụng, đo bằng thư viện `tracemalloc`.
- **Điều kiện dừng:**
 - Để đảm bảo thời gian thực nghiệm nằm trong giới hạn cho phép, các thuật toán vét cạn được thiết lập điều kiện dừng sớm dựa trên kích thước lưới:
 - * **Brute Force:** Bỏ qua nếu kích thước lưới $N > 7$.
 - * **Backtracking:** Bỏ qua nếu kích thước lưới $N > 13$.
 - Các trường hợp bị bỏ qua sẽ được gán giá trị NaN và không hiển thị trên biểu đồ (đường biểu diễn sẽ bị ngắt quãng).

4.3 Scalability Analysis

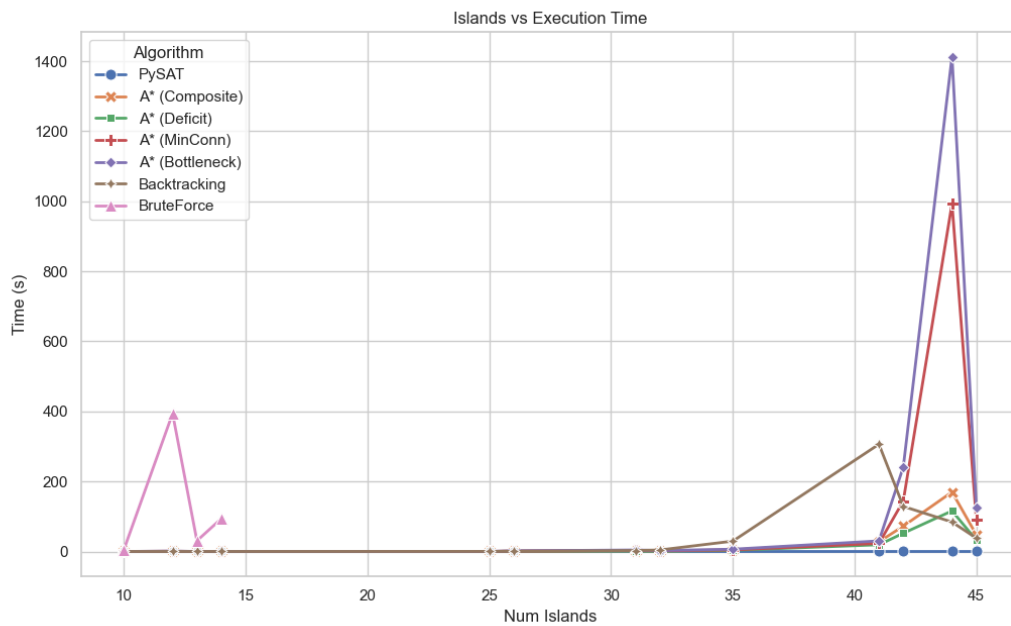
Phần này phân tích khả năng mở rộng của thuật toán khi kích thước bài toán tăng lên, dựa trên hai yếu tố chính: số lượng đảo và kích thước lưới.

4.3.1 Impact of Number of Islands

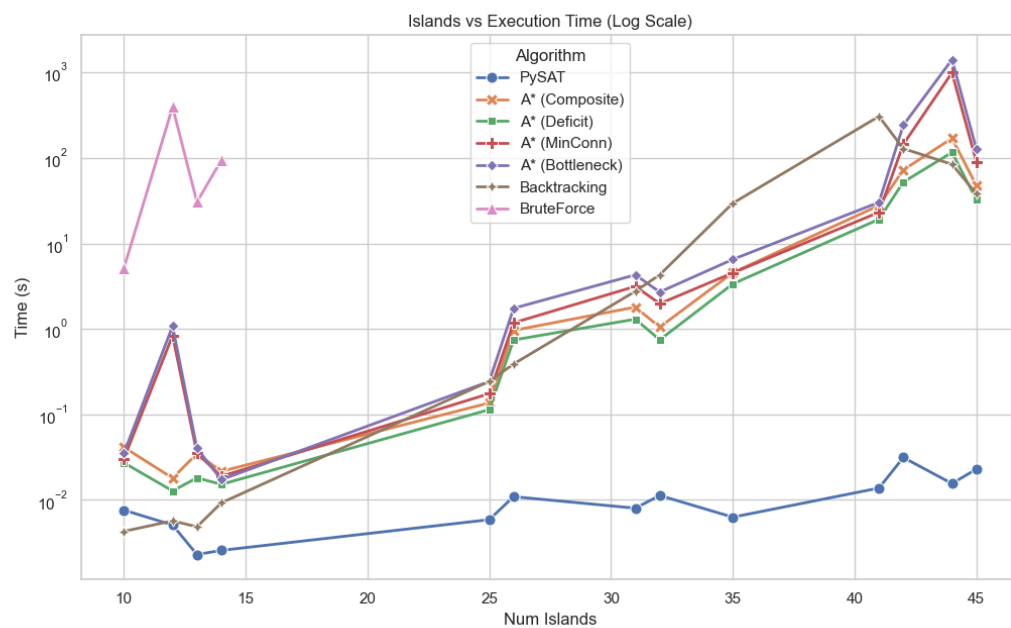
Số lượng đảo là yếu tố trực tiếp nhất ảnh hưởng đến độ phức tạp của bài toán Hashiwokakero. Hình 4.2 và 4.3 trình bày mối quan hệ giữa thời gian thực thi và số lượng đảo.

Phân tích chi tiết về thời gian thực thi:

- **Tăng trưởng hàm mũ:** Quan sát Hình 4.2, ta thấy thời gian chạy của tất cả các thuật toán đều tăng vọt khi số lượng đảo vượt quá ngưỡng 15-20. Điều này phản ánh đúng bản chất NP-đầy đủ của bài toán [3], nơi không gian trạng thái bùng nổ theo hàm mũ $O(b^d)$ (với b là hệ số nhánh và d là độ sâu, tỉ lệ thuận với số đảo).
- **Hiệu quả của Heuristic:** Trên biểu đồ Logarit (Hình 4.3), độ dốc của đường A^* (Composite) thấp hơn đáng kể so với Backtracking. Điều này chứng tỏ hàm Heuristic đã ước lượng tốt chi phí [4], giúp giảm cơ số của hàm mũ và cho phép thuật toán tiếp cận được các bài toán có kích thước lớn hơn.
- **Giới hạn của vét cạn:** Các đường biểu diễn của Brute Force và Backtracking bị ngắt quãng sớm (tại khoảng 15-20 đảo). Đây là kết quả của cơ chế "Skipping" được thiết lập trong thực nghiệm khi kích thước lưới vượt quá giới hạn chịu đựng ($N > 7$ hoặc $N > 13$), khẳng định sự không khả thi của các phương pháp vét cạn cho bài toán thực tế.



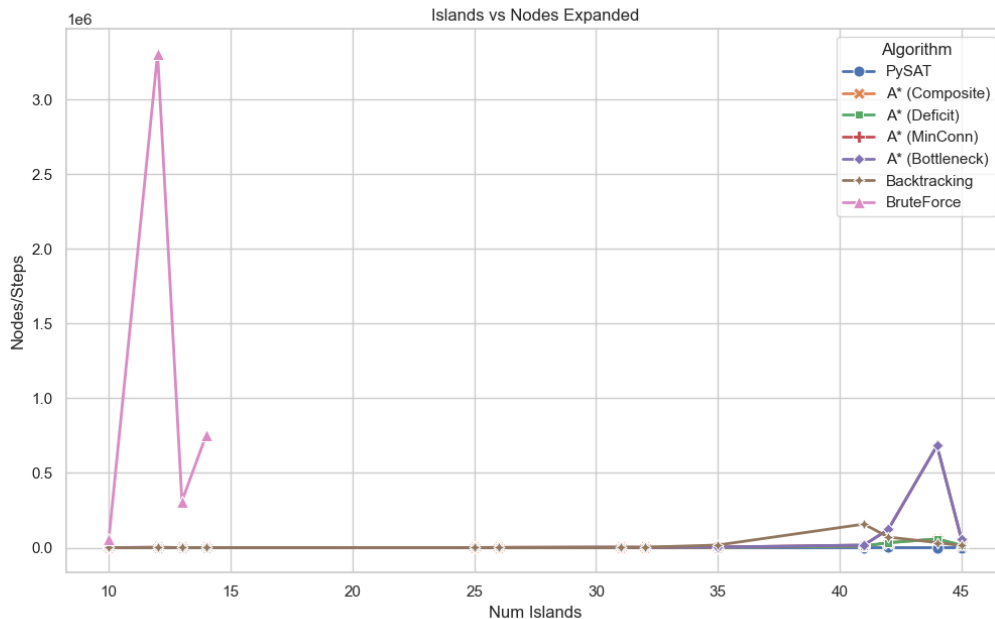
Hình 4.2: Thời gian thực thi theo số lượng đảo (Linear Scale)



Hình 4.3: Thời gian thực thi theo số lượng đảo (Log Scale)

- **PySAT Solver:** Bộ giải PySAT thể hiện hiệu năng vượt trội và ổn định ở các kích thước trung bình. Nhờ cơ chế lan truyền ràng buộc (Unit Propagation) và học mệnh đề (Clause Learning) của các SAT Solver hiện đại [5], phương pháp này thường tìm ra lời giải nhanh hơn A* trong các trường hợp không gian tìm kiếm quá sâu.

Bên cạnh thời gian, số lượng nút được duyệt phản ánh khả năng cắt tỉa không gian tìm kiếm của thuật toán.



Hình 4.4: Số lượng nút duyệt theo số lượng đảo

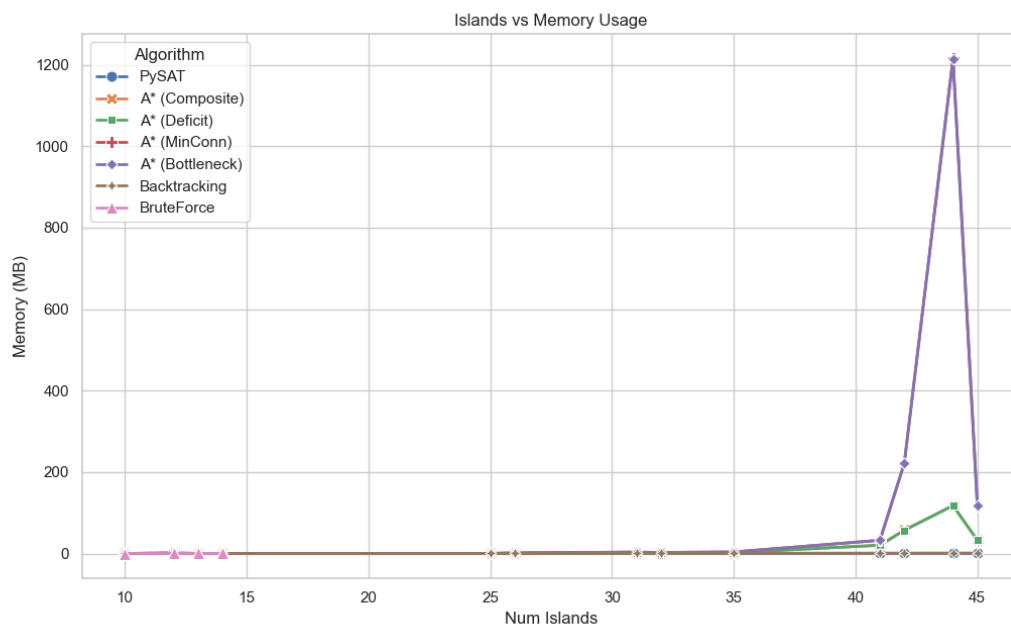
Phân tích về không gian tìm kiếm:

- **Sức mạnh của Pruning:** Hình 4.4 minh họa sự chênh lệch hàng triệu lần về số nút phải duyệt giữa các thuật toán. Trong khi Backtracking lãng phí tài nguyên vào các nhánh vô vọng, A* với Heuristic tốt tập trung mở rộng các nút tiềm năng, giảm thiểu đáng kể không gian tìm kiếm.
- **Vai trò của Forward Checking:** Biến thể Backtracking kết hợp Forward Checking cho thấy sự cải thiện rõ rệt so với phiên bản thuần túy. Việc phát hiện sớm các mâu thuẫn cục bộ (như đảo không còn đủ chỗ nối cầu) giúp cắt bỏ sớm các nhánh sai, ngăn chặn việc đi sâu vào các trạng thái không hợp lệ.

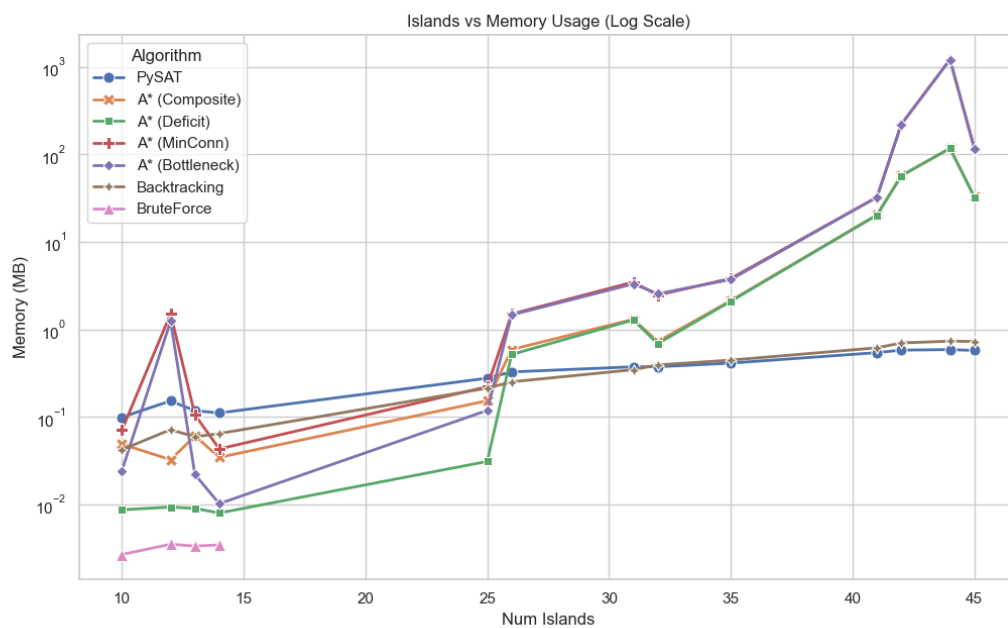
Về mặt bộ nhớ, Hình 4.5 và 4.6 minh họa chi phí lưu trữ.

Phân tích về bộ nhớ:

- **Time-Space Trade-off:** Đây là điểm khác biệt cốt lõi giữa DFS và BFS/A*. Backtracking (dựa trên DFS) duy trì mức tiêu thụ bộ nhớ thấp và ổn định $O(d)$. Ngược lại, A* phải lưu trữ toàn bộ tập Open và Closed để đảm bảo tính tối ưu, dẫn đến việc tiêu thụ bộ nhớ tăng nhanh theo số lượng đảo.
- **Memory Bottleneck:** Với các bài toán quy mô lớn (> 50 đảo), A* có xu hướng gặp lỗi tràn bộ nhớ trước khi hết thời gian cho phép. Điều này gợi ý rằng đối với các bản đồ siêu lớn, các thuật toán lai như IDA* (Iterative Deepening A*) [6] sẽ là hướng tiếp cận cần thiết để cân bằng lại yếu tố này.



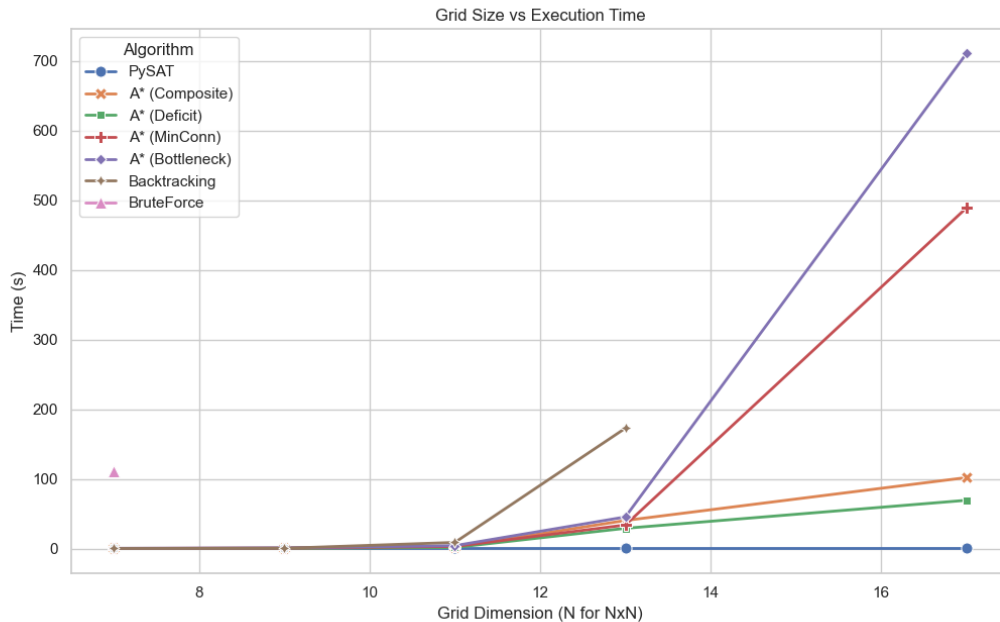
Hình 4.5: Bộ nhớ sử dụng theo số lượng đảo



Hình 4.6: Bộ nhớ sử dụng theo số lượng đảo (Log Scale)

4.3.2 Impact of Grid Size

Kích thước lưới (Grid Size) cũng ảnh hưởng đến độ phức tạp, tuy nhiên mức độ ảnh hưởng có thể khác so với số lượng đảo.



Hình 4.7: Thời gian thực thi theo kích thước lưới

Phân tích:

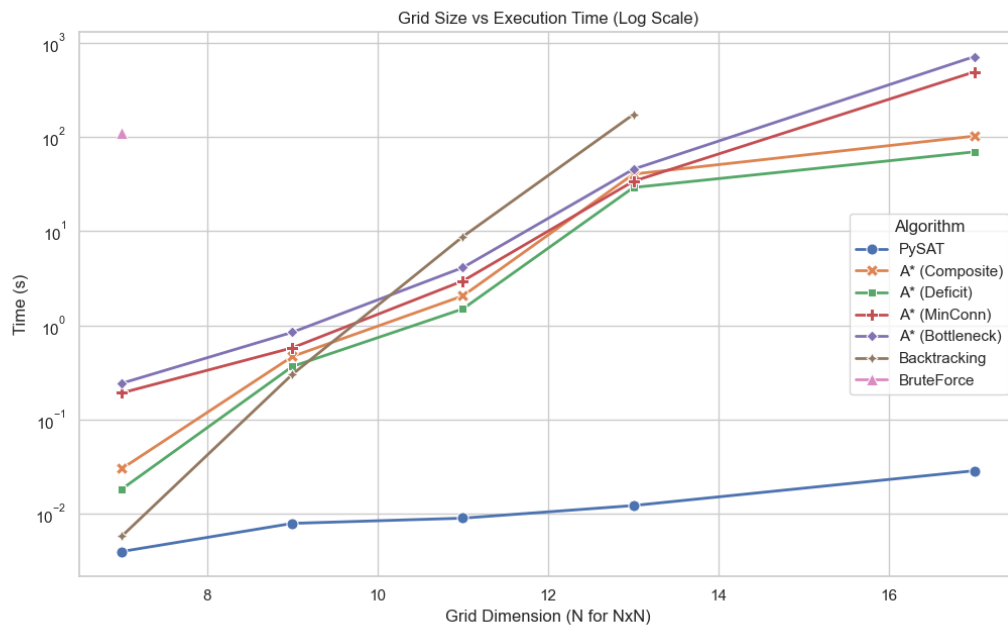
- **Island Density:** Khi kích thước lưới tăng nhưng số lượng đảo giữ nguyên, bài toán có thể trở nên dễ hơn do giảm bớt các ràng buộc chéo (crossing constraints). Tuy nhiên, không gian tìm kiếm vẫn mở rộng do độ dài đường đi tăng lên. **Mật độ đảo (số đảo / diện tích)** mới là chỉ số chính xác hơn để đánh giá độ khó thay vì chỉ dựa vào kích thước lưới đơn thuần.
- **High Variance:** Hình 4.7 cho thấy độ phân tán dữ liệu lớn hơn nhiều so với biểu đồ theo số đảo. Điều này chỉ ra rằng cấu trúc topo cụ thể của bản đồ (vị trí tương đối của các đảo) có ảnh hưởng quyết định đến độ khó. Một lưới nhỏ nhưng "chật chội" thường khó giải hơn một lưới lớn nhưng thưa thớt.
- **Cut-off Point:** Tương tự như phân tích trước, các thuật toán Brute Force và Backtracking nhanh chóng chạm ngưỡng giới hạn tại Grid Size $> 7 \times 7$ và 13×13 . Sân chơi ở các kích thước lớn hoàn toàn thuộc về A* và PySAT.

4.4 Heuristic Efficiency & Overall Performance

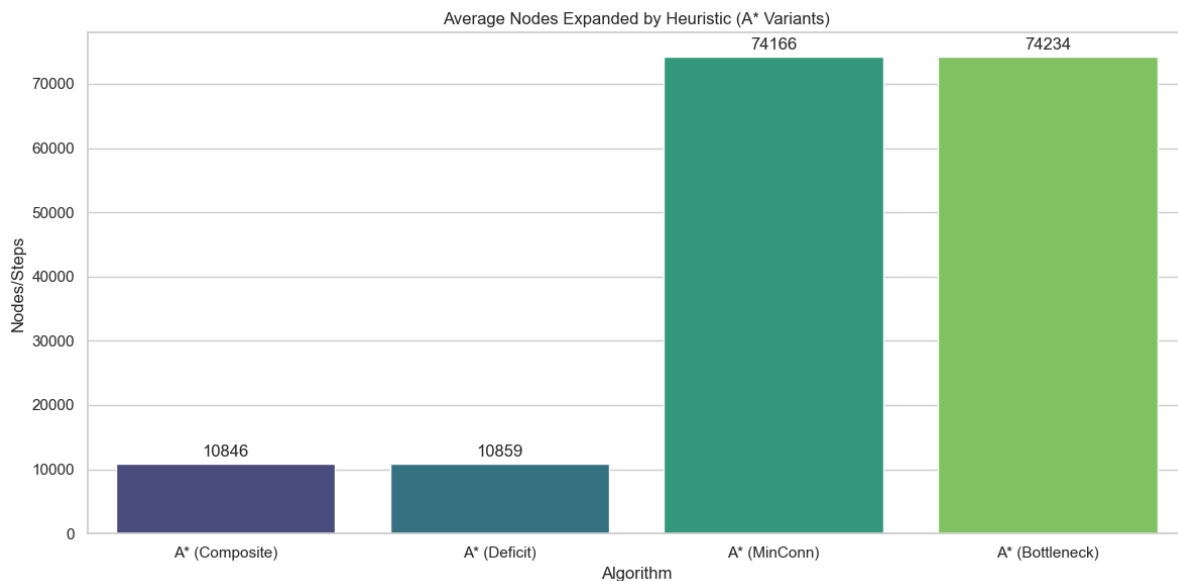
4.4.1 Comparison of Heuristics

Phần này so sánh hiệu quả của các hàm heuristic đã đề xuất: Deficit, Connectivity, Local Bottleneck và Composite. Mục tiêu là đánh giá sự đánh đổi giữa chi phí tính toán heuristic và khả năng cắt tỉa không gian tìm kiếm.

Nhận xét:

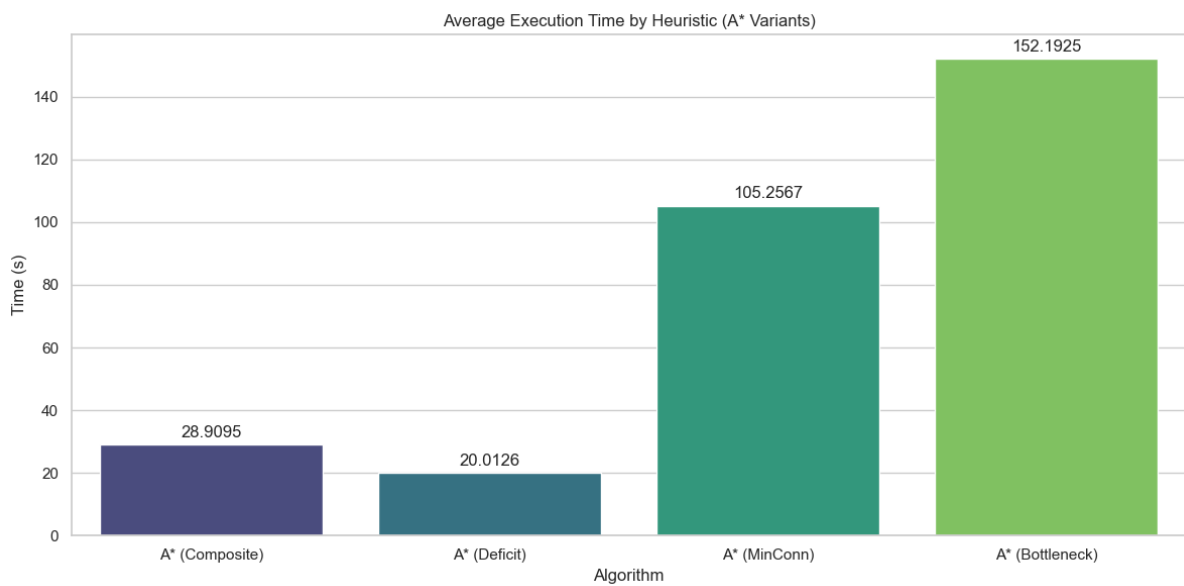


Hình 4.8: Thời gian thực thi theo kích thước lưới (Log Scale)



Hình 4.9: So sánh số nút duyệt giữa các Heuristic

- **Về số nút duyệt (Hình 4.9):** Composite Heuristic (kết hợp Max của các heuristic con) thể hiện sự vượt trội tuyệt đối về khả năng cắt tỉa. Số lượng nút duyệt luôn thấp nhất, khẳng định tính chất lý thuyết: heuristic càng chặt (càng gần giá trị thực h^*) thì không gian tìm kiếm càng được thu hẹp hiệu quả.
- **Về thời gian (Hình 4.10):** Tuy nhiên, Hình 4.10 cho thấy một thực tế thú vị: **Deficit Heuristic thường có tổng thời gian chạy nhanh nhất**. Lý do là chi phí tính toán của nó cực thấp (chỉ là phép trừ đơn giản). Trong khi đó, Composite Heuristic phải gánh chi phí tính toán của cả 3 heuristic con tại mỗi nút. Dù giảm

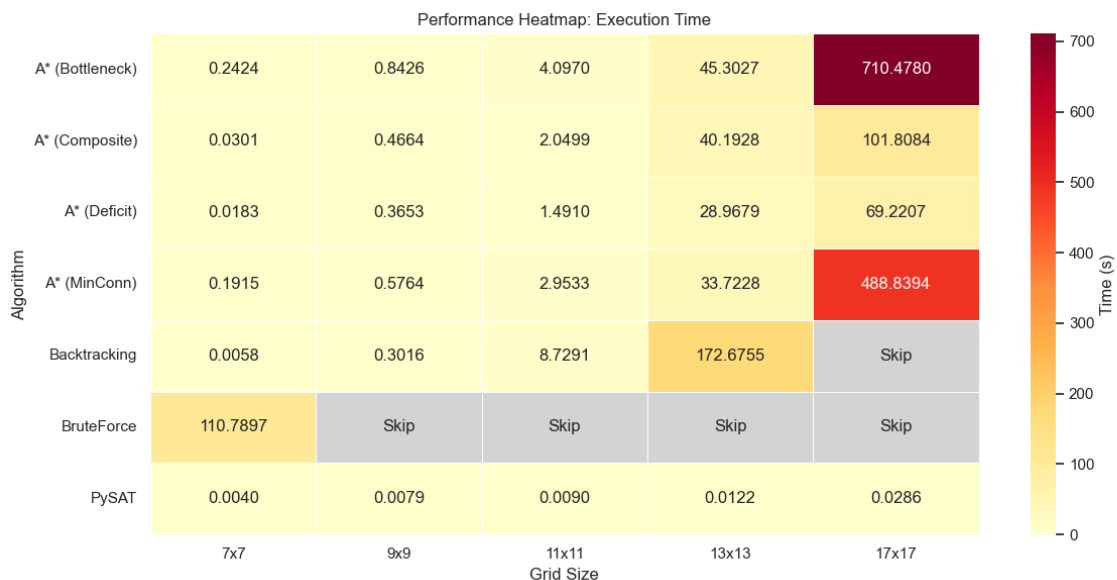


Hình 4.10: So sánh thời gian thực thi giữa các Heuristic

được số nút duyệt, nhưng "overhead" tính toán quá lớn khiến tổng thời gian của Composite thường chậm hơn so với Deficit trong các bộ test này.

4.4.2 Performance Heatmap

Hình 4.11 hiển thị ma trận so sánh hiệu năng giữa các thuật toán trên các kích thước lưới khác nhau.



Hình 4.11: Heatmap so sánh thời gian thực thi: Thuật toán vs Grid Size

Phân tích biểu đồ nhiệt:

- **Cấu trúc biểu đồ:** Trục tung liệt kê các thuật toán, trục hoành là kích thước lưới tăng dần. Màu sắc từ Vàng nhạt (nhanh) đến Đỏ (chậm). Các ô màu Xám thể hiện thuật toán bị bỏ qua (Skipped).

- **Cơ chế dừng:** Các dòng tương ứng với Brute Force và Backtracking chuyển sang màu Xám ngay từ các kích thước lưới nhỏ ($N > 7$ và $N > 13$). Đây là kết quả của việc **chủ động bỏ qua** các test case quá lớn trong mã nguồn benchmark để tránh thời gian chạy vô hạn, chứ không phải do lỗi runtime. Điều này phản ánh giới hạn thực tế của các thuật toán vét cạn.
- **Khả năng mở rộng của AI:** Ngược lại, các dòng tương ứng với A* và PySAT duy trì màu sắc trải dài đến tận kích thước 17×17 . Màu sắc chuyển dần từ Vàng sang Cam/Đỏ ở các cột bên phải cho thấy thời gian xử lý tăng lên theo độ khó, nhưng thuật toán vẫn tìm ra lời giải thành công (không bị Skip).

5 Conclusion

Đồ án này đã hoàn thành mục tiêu xây dựng và đánh giá các phương pháp giải quyết bài toán Hashiwokakero, từ các thuật toán tìm kiếm cơ bản như Backtracking, Brute Force đến các phương pháp nâng cao như A* Search và Logic-based (SAT Solver). Kết quả thực nghiệm trên bộ dữ liệu đa dạng đã khẳng định sự vượt trội của các phương pháp có tri thức bổ sung so với tìm kiếm mù. Cụ thể, thuật toán A* với Composite Heuristic đã chứng minh khả năng cắt tỉa không gian tìm kiếm cực kỳ hiệu quả, giảm thiểu số lượng nút duyệt xuống hàng triệu lần so với Backtracking, cho phép giải quyết các bài toán có kích thước lớn mà các phương pháp cổ điển không thể chạm tới.

Bên cạnh đó, việc áp dụng SAT Solver thông qua thư viện PySAT đã mở ra một hướng tiếp cận đầy tiềm năng. Bằng cách mô hình hóa bài toán dưới dạng các mệnh đề logic CNF, chúng ta có thể tận dụng sức mạnh của các bộ giải SAT hiện đại [5] để tìm lời giải một cách nhanh chóng và ổn định. Thực tế cho thấy, trong nhiều trường hợp kích thước trung bình, PySAT thậm chí còn cho tốc độ xử lý tốt hơn cả A*, đồng thời tránh được vấn đề bùng nổ bộ nhớ thường gặp ở các thuật toán tìm kiếm dựa trên đồ thị.

Tuy nhiên, nghiên cứu cũng chỉ ra những giới hạn không thể tránh khỏi của bài toán NP-đầy đủ. Dù A* và SAT Solver rất mạnh mẽ, chúng vẫn đối mặt với rào cản về tài nguyên tính toán khi kích thước bài toán tăng lên quá lớn (trên 50 đảo hoặc lưới siêu rộng). Sự đánh đổi giữa thời gian và không gian (Time-Space Trade-off) thể hiện rõ rệt nhất ở A*, khi tốc độ tìm kiếm nhanh phải trả giá bằng việc tiêu tốn lượng lớn bộ nhớ RAM. Nhìn chung, đồ án đã cung cấp một cái nhìn toàn diện về hiệu năng của các thuật toán, từ đó đưa ra cơ sở để lựa chọn phương pháp phù hợp tùy theo quy mô và yêu cầu cụ thể của bài toán thực tế.

6 Tài liệu

- [1] GeeksforGeeks. Constraint satisfaction problems (CSP) in artificial intelligence, 9 2024.
- [2] GeeksforGeeks. Prim's minimum spanning tree (MST) | greedy algo-5, 11 2024.
- [3] Daniel Andersson. Hashiwokakero is np-complete. *Information Processing Letters*, 109(19):1143–1146, 2009.

- [4] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [5] Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh. *Handbook of satisfiability*, volume 185. IOS press, 2009.
- [6] Richard E Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial intelligence*, 27(1):97–109, 1985.